SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE:** CSA3711

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Code of Conduct:

I S. Kamalesh (192521013) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Kamalesh

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Question 1 *(Bloom's Level 5)*

An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.

Answer – Online Banking – Fund Transfer Boundary Failures

- Failures cluster at the minimum and maximum transfer limits, which shows the current suite never targets the edges of the valid range. Random inputs almost never land exactly on a limit.
- Random testing gives no coverage guarantee, cannot be repeated reliably and spends effort on redundant mid-range values.
- Redesign: Equivalence Class Partitioning (ECP) splits the amount into valid and invalid classes; Boundary Value Analysis (BVA) adds the values on and next to each limit. Assumed limits: minimum Rs. 100, maximum Rs. 1,00,000.
- Decision: replace random testing as the primary technique and keep only a small random batch as a supplement for unexpected inputs.

Table 1.1 – ECP + BVA test cases (assumed limits Rs. 100 – Rs. 1,00,000)

TC	Class / Boundary	Input (Rs.)	Expected result
TC1	Invalid – just below minimum	99	Rejected – below minimum
TC2	Minimum boundary	100	Accepted
TC3	Minimum + 1	101	Accepted
TC4	Valid – nominal	50,000	Accepted
TC5	Maximum – 1	99,999	Accepted
TC6	Maximum boundary	1,00,000	Accepted
TC7	Invalid – just above maximum	1,00,001	Rejected – limit exceeded
TC8	Invalid – zero / negative	0, -500	Rejected
TC9	Invalid – non-numeric / blank	abc, (blank)	Rejected – validation message
TC10	Valid amount, low balance	1,00,000 (balance 90,000)	Rejected – insufficient funds

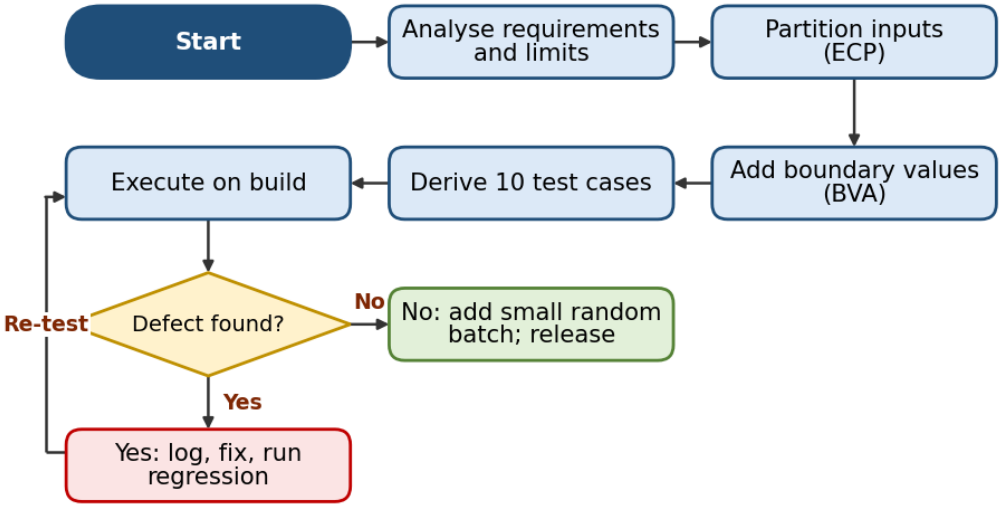


Figure 1.1 – Structured test design flow for fund transfer

Table 1.2 – Random testing versus the proposed ECP + BVA approach

Criterion	Random testing	ECP + BVA (proposed)
Chance of hitting a boundary	Very low	Guaranteed by design
Repeatability	Poor	High – fixed, documented cases
Number of tests	Large and redundant	10 focused cases
Coverage measurable?	No	Yes – class and boundary coverage
Defect detection at limits	Unreliable	Maximum
Verdict	Retain only as a supplement	Primary technique

Conclusion: ECP + BVA directly targets the limit values where the failures occur, so it detects the defects with far fewer, repeatable tests than random testing.

Question 2 *(Bloom's Level 5)*

A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.

Answer – Healthcare Module – Insufficient Coverage and CFG

- Critical defects escaped because the suite exercised only typical paths. A conceptual dose-calculation module is used here: read inputs, check validity (D1), allergy (D2), age below 12 (D3) and renal impairment (D4), then output the dose.
- With 4 decisions there are 8 branches and 9 paths (1 error path + 2 x 2 x 2 valid paths). The existing suite, illustrated by 2 tests (adult and paediatric, no allergy, normal kidney function), covers only a small part of this.
- Improved strategy: derive one test for every independent path, set the adequacy criterion to 100% branch coverage and complete path coverage of the 9 paths (feasible because the module is small).

- This raises reliability because rare but critical routes (allergy, renal reduction, invalid input) are now verified before deployment.

Table 2.1 – Coverage achieved by the existing suite (illustrative)

Criterion	Total	Covered	Achieved	Status
Statement coverage	7	4	57%	Partial – S2, S3, S6 missed
Branch coverage	8	5	62.5%	Partial – D1, D2, D4 true outcomes missed
Path coverage	9	2	22%	Not covered – 7 paths untested

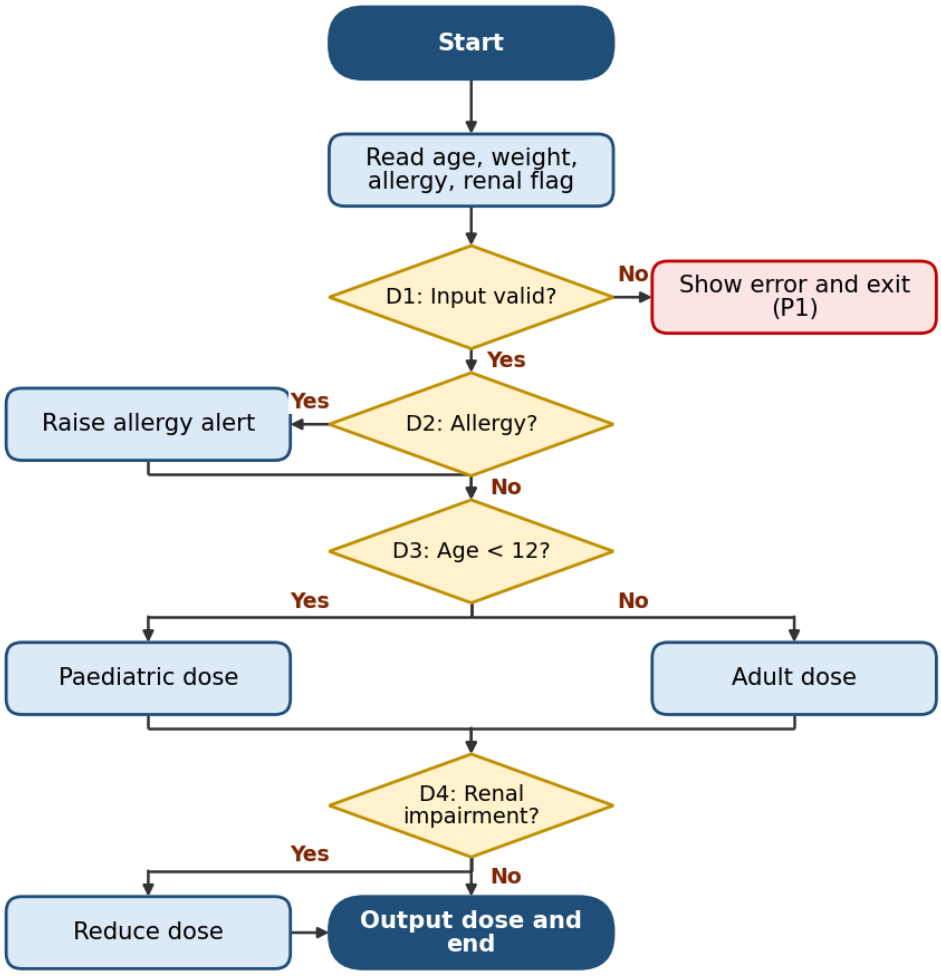


Figure 2.1 – Conceptual control flow graph of the dose calculation module

Table 2.2 – Improved test set (one test per path)

Path	D1	D2	D3	D4	Test scenario
P1	N	–	–	–	Invalid age / weight – error message
P2	Y	N	Adult	N	Adult, no allergy, normal kidney (existing)
P3	Y	N	Child	N	Child, no allergy, normal kidney (existing)
P4	Y	N	Adult	Y	Adult with renal impairment – reduced dose
P5	Y	N	Child	Y	Child with renal impairment – reduced dose
P6	Y	Y	Adult	N	Adult with allergy – alert shown
P7	Y	Y	Child	N	Child with allergy – alert shown
P8	Y	Y	Adult	Y	Adult, allergy and renal impairment
P9	Y	Y	Child	Y	Child, allergy and renal impairment

Conclusion: Mapping every decision to the CFG exposes the 7 untested paths; testing all 9 gives full statement, branch and path coverage and lowers post-deployment risk.

Question 3 *(Bloom's Level 5)*

A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.

Answer – E-commerce – Price, Quantity and Discount Combinations

- The current approach tests each field mostly in isolation, so errors that appear only for particular combinations of price, quantity and discount are never triggered.
- ECP and BVA give strong coverage for a single field but do not describe how fields interact; a decision table is needed for combinations.
- Assumed rules: price 0.01 – 99,999.99; quantity 1 – 100; discount 0 – 50%. Order total = price x quantity x (1 – discount).
- Revised strategy: ECP + BVA per field, then a decision table over the validity of all three fields (2 x 2 x 2 = 8 rules) so every combination class is covered.

Table 3.1 – ECP and BVA per input field

Field	Valid class	Invalid classes	Boundary test values
Price	0.01 – 99,999.99	<= 0, non-numeric, blank	0, 0.01, 0.02, 99,999.98, 99,999.99, 1,00,000
Quantity	1 – 100	0, negative, > 100, decimal	0, 1, 2, 99, 100, 101
Discount (%)	0 – 50	< 0, > 50	-1, 0, 1, 49, 50, 51

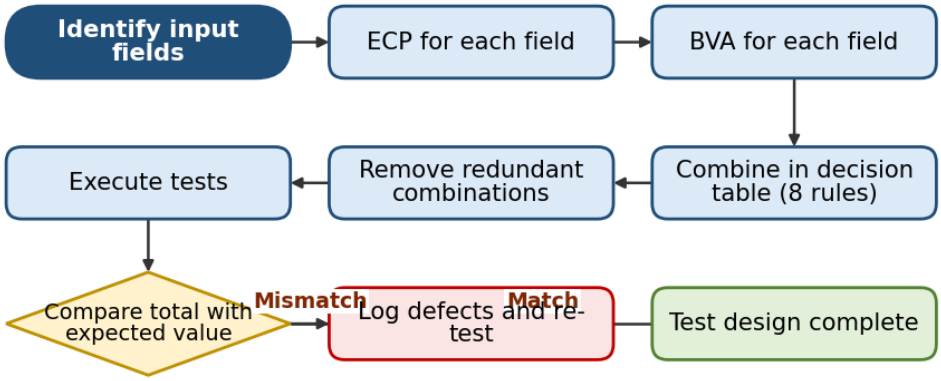


Figure 3.1 – Revised black box test design flow

Table 3.2 – Decision table covering all input combinations

Rule	Price valid	Qty valid	Discount valid	Expected action
R1	Y	Y	Y	Compute total correctly
R2	Y	Y	N	Reject discount, show message
R3	Y	N	Y	Reject quantity
R4	Y	N	N	Reject quantity and discount
R5	N	Y	Y	Reject price
R6	N	Y	N	Reject price and discount
R7	N	N	Y	Reject price and quantity
R8	N	N	N	Reject all fields

Conclusion: Field-level ECP/BVA plus an 8-rule decision table covers every input combination class, improving defect detection and test completeness.

Question 4 *(Bloom's Level 5)*

A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.

Answer – Nested Conditions and Loops – Path Coverage Gaps

- Module under analysis (conceptual): a loop reads n numbers; for each positive number it counts evens or odds, otherwise counts negatives. After the loop, the result is OK only if (evens > odds AND negatives = 0).
- Gaps found in the white box approach: the compound condition was drawn as a single node in the CFG, the loop was only tested with typical sizes, and no test forced the loop to run zero times.
- Predicates: while loop (1), a[i] > 0 (1), even check (1), compound final condition (2) = 5, so V(G) = P + 1 = 6 independent paths are needed. Missing even one of them leaves part of the logic untested.
- Strategy: draw a complete CFG, calculate V(G), and write one test for each independent path (basis path testing).

Table 4.1 – Gaps in the current testing and their impact

Gap	Effect on defect detection
Compound condition shown as one node	Condition 'neg = 0' never tested on its own
Loop executed only with typical sizes	Zero-iteration and multi-iteration defects missed
No test with negative or odd data	Else branches never executed
Independent paths not identified	Only partial path coverage reported

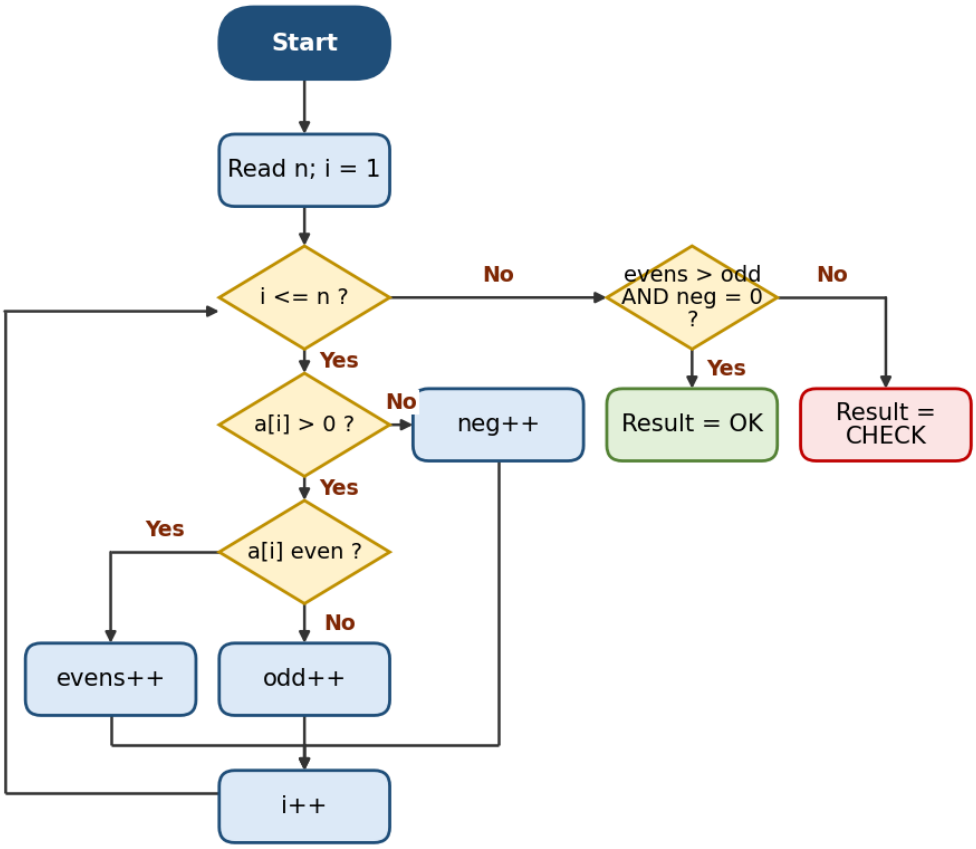


Figure 4.1 – Control flow graph of the nested module

Table 4.2 – Independent paths (V(G) = 6) and test data

Path	Description	Test data a[ ]	Expected
P1	Loop not entered (n = 0)	(empty)	CHECK
P2	One positive even value	[4]	OK
P3	One positive odd value	[3]	CHECK
P4	One negative value	[-2]	CHECK
P5	evens > odd but neg <> 0	[4, -1]	CHECK
P6	Loop with several iterations	[4, 6]	OK

Conclusion: Using the CFG to find all 6 independent paths ensures every decision outcome and loop behaviour is tested, giving full branch and basis-path coverage.

Question 5 *(Bloom's Level 5)*

A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.

Answer – Financial Application – Hybrid Black Box and White Box Strategy

- Defects persist because the two techniques were applied separately: black box tests were not linked to code structure and white box tests did not use specification boundaries. Coverage metrics were reported but not used as exit criteria.
- Test adequacy was judged by the number of tests rather than by criteria such as boundary coverage, branch coverage and path coverage.
- Proposed hybrid: ECP and BVA derive tests from the specification (for example loan amount, tenure, interest rate); CFG and basis path testing derive tests from the code (EMI, penalty and interest calculation); both sets are merged and measured against defined adequacy criteria.
- Defects are prevented earlier because gaps found by coverage measurement are fed back into new tests before release.

Table 5.1 – Analysis of the existing strategy

Area	Observed problem	Root cause
Test design	Black box and white box suites overlap in some areas and leave others empty	No common traceability between requirements and code
Adequacy criteria	Number of test cases used as the measure	No boundary, branch or path targets set
Coverage metrics	Statement coverage reported, branch and path not measured	Metrics not used as release criteria
Production defects	Boundary and rare-path failures	Neither technique covered the full logic

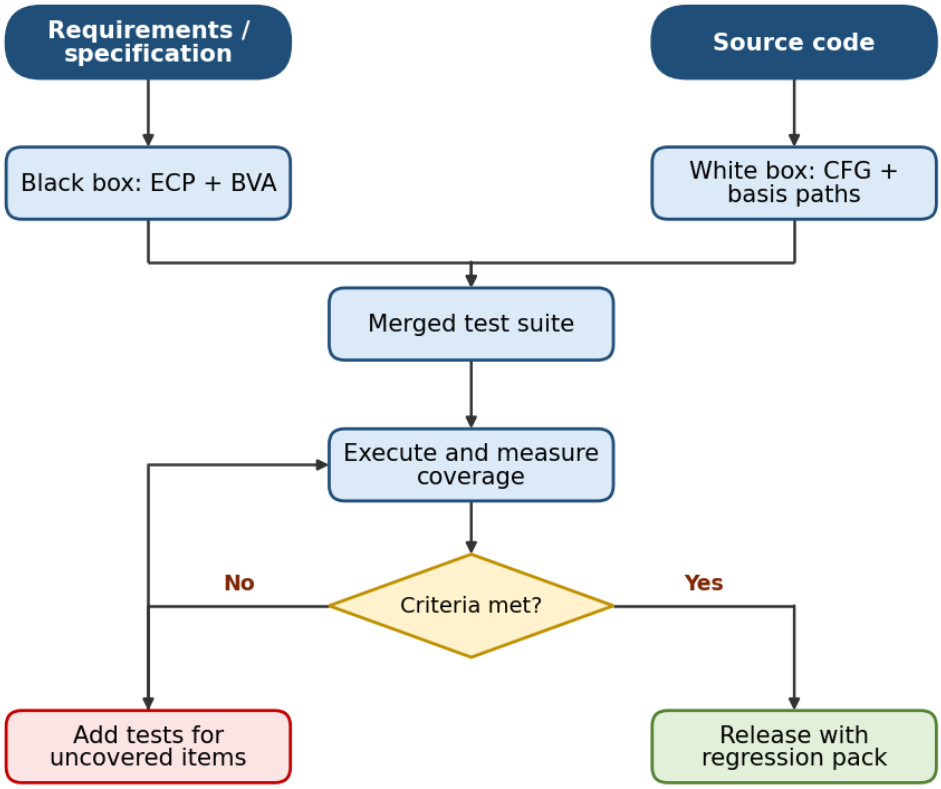


Figure 5.1 – Hybrid test design process

Table 5.2 – Hybrid strategy and adequacy criteria

Layer	Technique	Applied to	Adequacy target
Black box	ECP	Amount, tenure, rate classes	All classes covered
Black box	BVA	Limits of each input	100% boundary values
White box	CFG construction	Interest and EMI modules	Complete graph for each module
White box	Basis path testing	Penalty and settlement logic	All V(G) paths
Combined	Coverage measurement	Whole module	100% branch coverage

Conclusion: Combining specification-based and code-based tests, controlled by explicit adequacy criteria, gives better coverage and prevents the defects that either technique alone misses.

Question 6 *(Bloom's Level 5)*

A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.

Answer – HR Payroll – Incorrect Bonus at 5, 10 and 15 Years

- Employees exactly at 5, 10 and 15 years were wrongly paid, which is a typical off-by-one boundary defect such as using '>' instead of '>=' in the service-band check. The test design clearly did not include boundary values.

- Assumed rule: 0-4 years = 5% bonus, 5-9 = 10%, 10-14 = 15%, 15 and above = 20%; performance rating 1-5 adjusts the bonus.
- ECP gives the service bands and rating classes; BVA tests the value below, on and above every band edge (4/5/6, 9/10/11, 14/15/16).
- Exploratory testing depends on tester intuition and is not repeatable, whereas BVA/ECP is systematic, documented and guarantees that financial thresholds are tested.

Table 6.1 – BVA test values for years of service (assumed bonus rates)

TC	Years of service	Class	Expected bonus
TC1	-1	Invalid	Rejected
TC2	0	Band 0-4 (minimum)	5%
TC3	4	Band 0-4 (maximum)	5%
TC4	5	Band 5-9 (lower boundary)	10%
TC5	6	Band 5-9	10%
TC6	9	Band 5-9 (upper boundary)	10%
TC7	10	Band 10-14 (lower boundary)	15%
TC8	14	Band 10-14 (upper boundary)	15%
TC9	15	Band 15+ (lower boundary)	20%
TC10	16	Band 15+	20%

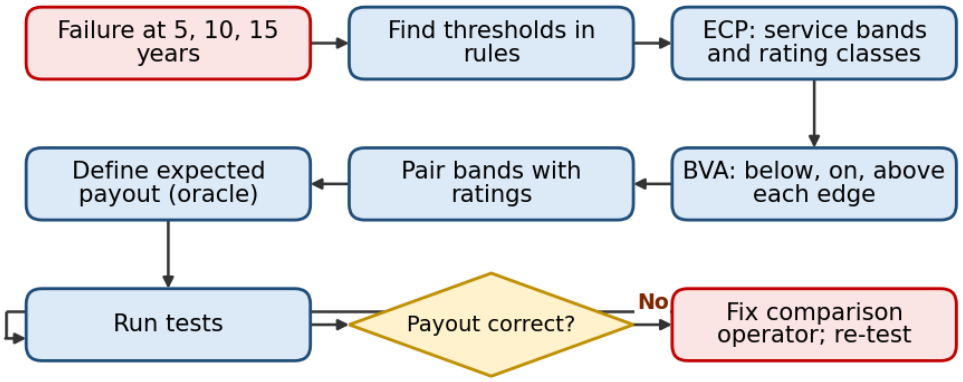


Figure 6.1 – Redesign of the payroll test suite

Table 6.2 – BVA/ECP versus exploratory testing for payroll logic

Aspect	Exploratory testing	BVA + ECP
Boundary coverage	Depends on tester's guess	Systematic at every threshold
Repeatability	Low	High – can be re-run after every fix
Evidence for audit	Limited notes	Documented cases with expected payouts
Cost of a payout error	High (money is affected)	Prevented by design
Suitability for exact rules	Poor	Very good

Conclusion: BVA with ECP finds the exact threshold errors and is repeatable and auditable, so it is superior to exploratory testing for deterministic financial rules.

Question 7 *(Bloom's Level 5)*

An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.

Answer – Laboratory Temperature Controller – CFG and MC/DC

- The alarm failed in one sensor-malfunction sequence. Assume the alarm decision is D = (S AND NOT B) OR T, where S = sensor fault, B = backup sensor OK and T = over-temperature.
- The old tests reached the alarm only through T = 1. Decision (branch) coverage was therefore satisfied, but the combination S = 1 with B = 0 was never exercised, so the defect stayed hidden.
- MC/DC requires every condition to be shown to independently change the decision outcome; for 3 conditions this needs n + 1 = 4 tests.
- In safety-critical software, MC/DC removes masking of conditions and gives high confidence that every alarm trigger works.

Table 7.1 – Branch coverage gaps in the existing suite

Branch	Description	Existing tests	Status
B1	Temperature < low limit: heater ON	Yes	Covered
B2	Temperature > high limit: cooler ON	Yes	Covered
B3	Temperature normal: idle	Yes	Covered
B4	Over-temperature (T = 1): alarm	Yes	Covered
B5	Sensor fault with backup failed (S AND NOT B): alarm	No	Not covered
B6	Sensor fault with backup OK: switch to backup	No	Not covered

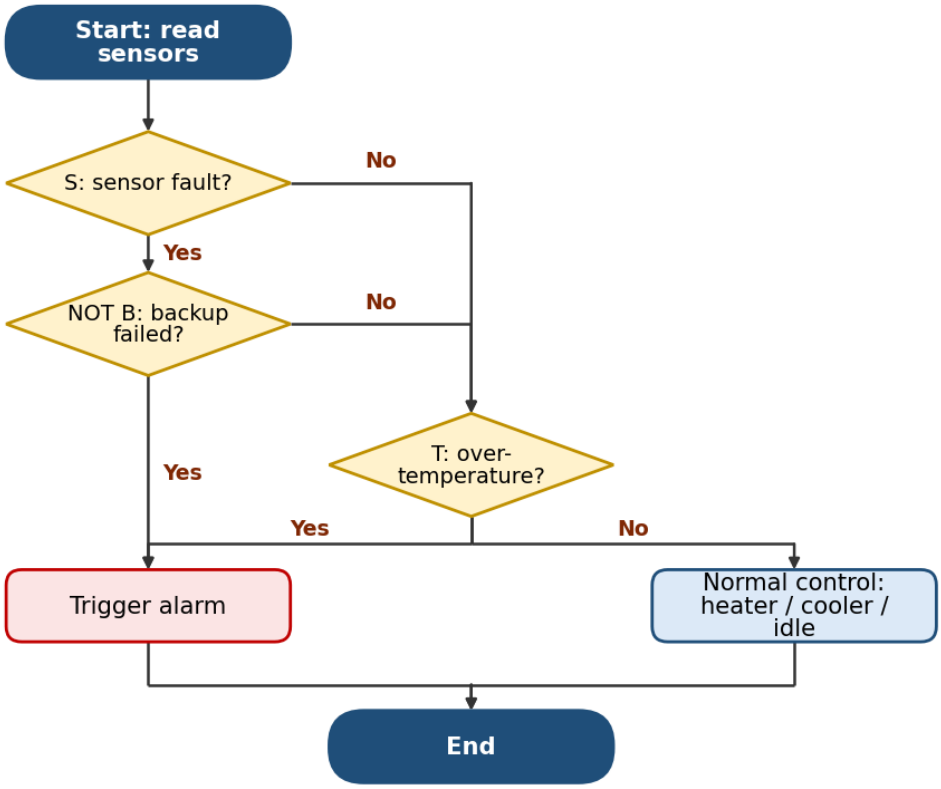


Figure 7.1 – CFG of the alarm decision with condition-level nodes

Table 7.2 – MC/DC test set for D = (S AND NOT B) OR T

TC	S	B	T	D	Independence pair shown
T1	0	0	0	0	Base case
T2	1	0	0	1	T1 and T2 – S changes D
T3	1	1	0	0	T2 and T3 – B changes D
T4	0	0	1	1	T1 and T4 – T changes D

Conclusion: Four MC/DC tests prove that each condition independently controls the alarm, closing the sensor-malfunction gap that branch coverage could not detect.

Question 8 *(Bloom's Level 5)*

A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.

Answer – Travel Portal – Promo Code Combinations

- Testing promo codes one at a time cannot reveal defects that occur only when two promos, a user tier and the ticket price interact, which is how a negative total appears.
- Decision table testing lists the conditions (percentage promo, flat promo, premium tier, discount not above price) and the resulting action for every meaningful combination.
- Assumed rules: only Premium users may stack two promos; the payable total must never be below Rs. 0.

- Don't-care entries merge rules, so the 16 raw combinations reduce to 7 rules and 7 tests with no redundancy.

Table 8.1 – Decision table (Y = yes, N = no, – = don't care)

Rule	% promo	Flat promo	Premium	Discount <= price	Action
R1	N	N	–	–	Charge full price
R2	Y	N	–	Y	Apply % discount
R3	N	Y	–	Y	Apply flat discount
R4	N	Y	–	N	Cap total at Rs. 0, show warning
R5	Y	Y	N	–	Reject second promo
R6	Y	Y	Y	Y	Apply both promos
R7	Y	Y	Y	N	Cap total at Rs. 0, show warning

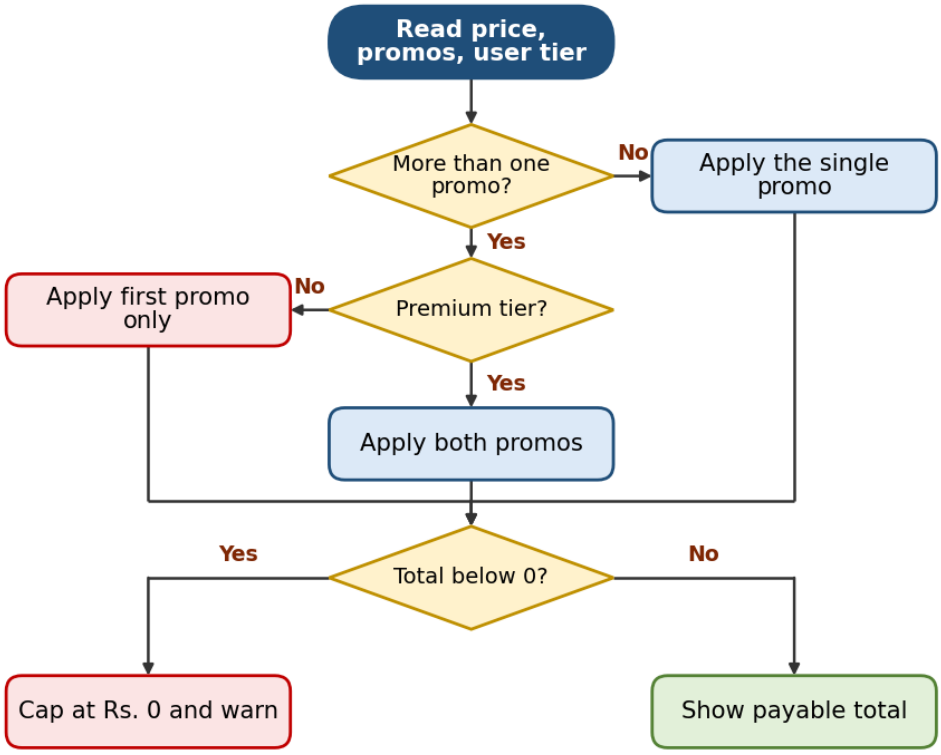


Figure 8.1 – Promo code evaluation flow

Table 8.2 – Optimized test set (price Rs. 5,000 unless stated)

TC	Input	Tier	Expected total	Rule
T1	No promo	Standard	5,000	R1
T2	20% promo	Standard	4,000	R2
T3	Flat Rs. 1,000 promo	Standard	4,000	R3
T4	Price Rs. 800, flat Rs. 1,000	Standard	0 and warning	R4
T5	20% + flat Rs. 1,000	Standard	Second promo rejected	R5
T6	20% + flat Rs. 1,000	Premium	3,000	R6
T7	Price Rs. 1,000, 50% + flat Rs. 800	Premium	0 and warning	R7

Conclusion: The decision table maps every valid and invalid promo combination; seven tests give complete coverage with no redundant cases and expose the negative-total defect.

Question 9 *(Bloom's Level 5)*

A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.

Answer – Database Migration Tool – Nested Loops and Infinite Loop Crashes

- Statement coverage only proves each line ran at least once. A test in which all records are valid executes every statement, yet it never takes the 'invalid record' branch where the loop counter is not incremented, so the infinite loop stays hidden.
- Statement coverage also ignores loop boundaries (zero, one, many iterations) and the number of independent paths.
- Predicates: outer loop (1), inner loop (1), valid record (1), duplicate record (1) = 4. Cyclomatic complexity V(G) = P + 1 = 5, equal to E - N + 2 for the CFG, so 5 independent paths must be tested.
- Strategy: basis path testing for the 5 paths, plus loop boundary tests with 0, 1, 2, n-1 and n iterations and a timeout guard on every test.

Table 9.1 – Why statement coverage was insufficient

Criterion	What it proves	Finds the infinite loop?
Statement coverage	Each line executed once	No – all-valid data hides it
Branch coverage	Both outcomes of 'valid record?' taken	Yes
Loop boundary testing	0, 1, 2, n-1, n iterations	Yes
Basis path testing (V(G) = 5)	Every independent path executed	Yes

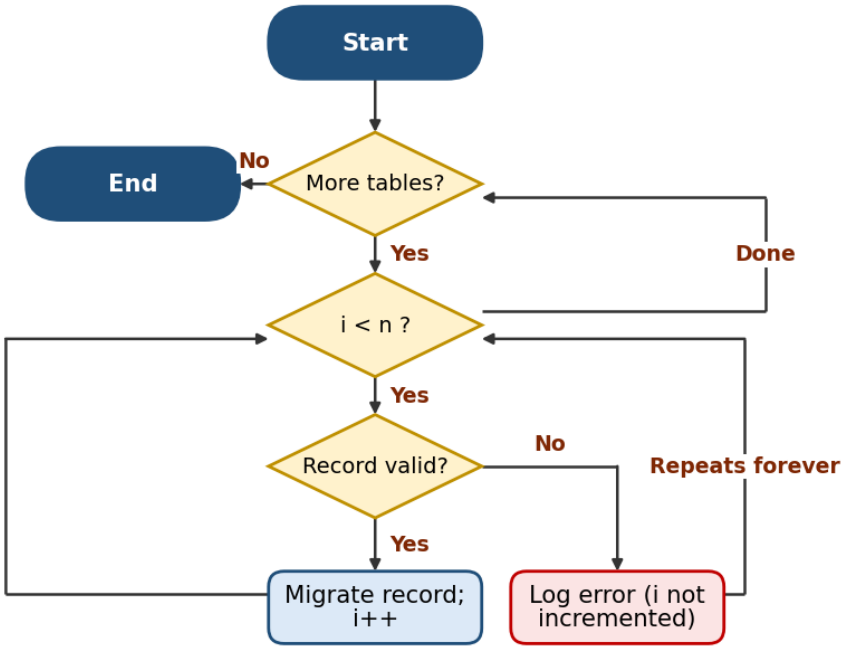


Figure 9.1 – CFG of the validation loop (the bug is the missing increment)

Table 9.2 – Independent paths (V(G) = 5)

Path	Scenario	Expected result
P1	No tables – outer loop skipped	Ends immediately
P2	Table with 0 records – inner loop skipped	Next table processed
P3	All records valid	Migrated, loop ends
P4	One invalid record	Error logged, loop must still end (timeout reveals the bug)
P5	Duplicate record	Skipped without a crash

Conclusion: V(G) = 5 gives the minimum set of paths to test; including the invalid-record path exposes the infinite loop that statement coverage cannot.

Question 10 *(Bloom's Level 5)*

A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.

Answer – Food Delivery App – State Transition Testing

- The tracker moves through Ordered, Preparing, Out for Delivery and Delivered. The design has no transition for a driver who cancels mid-delivery, so the app stays in 'Out for Delivery' and appears stuck.
- ECP and BVA test input values; they do not test sequences of events or whether an event is allowed in a given state. State transition testing checks every state, event and transition, including invalid ones.
- Redesign: add a 'Reassigning driver' state (new driver accepts or timeout leads to Cancelled with refund) and test each transition, both valid and invalid.
- This closes logic gaps that depend on history and order of events.

Table 10.1 – State transition table (redesigned)

Current state	Event	Next state	Valid?
Ordered	Restaurant accepts	Preparing	Valid
Ordered	Customer cancels	Cancelled	Valid
Preparing	Driver picks up order	Out for Delivery	Valid
Preparing	Customer cancels	Cancelled (refund)	Valid
Out for Delivery	Customer confirms delivery	Delivered	Valid
Out for Delivery	Driver cancels	Reassigning driver	Missing in old design
Reassigning driver	New driver accepts	Out for Delivery	Missing in old design
Reassigning driver	No driver within timeout	Cancelled (refund)	Missing in old design
Delivered	Any further event	No change	Invalid
Ordered	Delivery confirmed	No change	Invalid

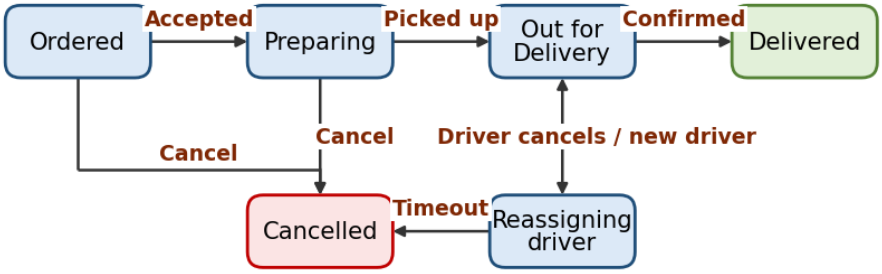


Figure 10.1 – State transition diagram

Table 10.2 – Test scenarios from the state model

TC	Event sequence	Expected final state
T1	Accept – pick up – confirm (happy path)	Delivered
T2	Customer cancels in Ordered	Cancelled
T3	Driver cancels during Out for Delivery	Reassigning driver
T4	T3, then new driver accepts	Out for Delivery
T5	T3, then no driver within timeout	Cancelled with refund
T6	Any event after Delivered	Stays Delivered
T7	Delivery confirmed while in Ordered	Rejected, state unchanged

Conclusion: State transition testing covers whole event sequences, including driver cancellation and invalid events, which ECP and BVA cannot represent.

Question 11 *(Bloom's Level 5)*

A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.

Answer – Authentication Module – Hybrid Testing for Brute Force

- The module combines password strength, OTP timing and IP location. Functional tests confirmed correct logins but did not attack the logic, so missing lockouts and attempt limits were not noticed.
- Error guessing uses tester experience of known attacks (rapid password guesses, OTP enumeration, expired OTP reuse, IP changes) to design negative tests.
- Formal path analysis then lists every route through the authentication logic so each failure and lockout route has a test.
- Combining both finds the weak points quickly and proves that none of the logic paths was left out, which strengthens the security posture.

Table 11.1 – Error guessing attack scenarios

Guess	Test input	Secure behaviour expected
Password brute force	100 wrong passwords in 1 minute	Lock after 5 attempts
OTP enumeration	Try 000000 to 999999	Lock after 3 wrong OTPs
Expired OTP	Enter OTP after 31 s (limit 30 s)	Rejected
OTP reuse	Reuse a used OTP	Rejected
Common password	Password@123	Blocked by strength rule
IP change mid-login	Different country after password	Step-up check or block

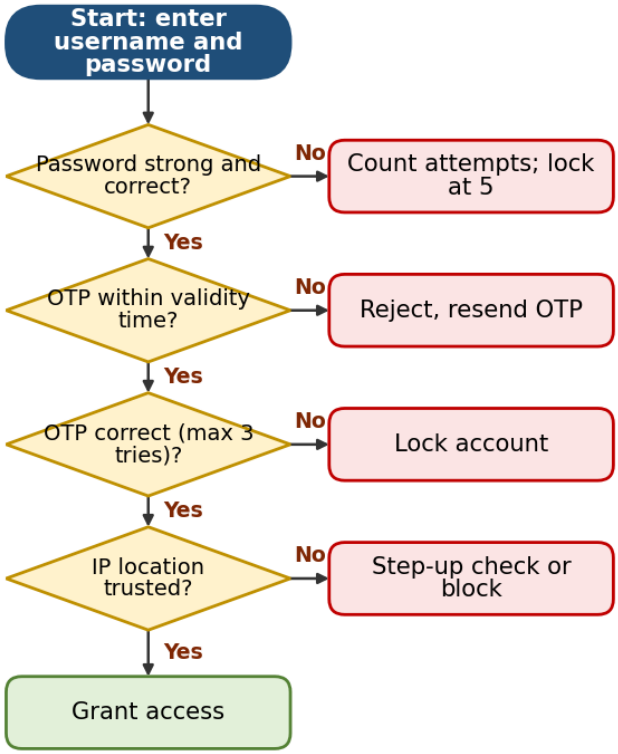


Figure 11.1 – Authentication flow analysed for paths

Table 11.2 – Path analysis of the authentication logic

Path	Conditions	Outcome	Test type
P1	Weak or wrong password, attempts < 5	Rejected	Black box
P2	Wrong password, attempts >= 5	Account locked	Error guessing
P3	Password OK, OTP expired	Rejected	Boundary + error guessing
P4	OTP wrong 3 times	Account locked	Error guessing
P5	OTP OK, IP not trusted	Step-up or block	Error guessing
P6	All checks pass	Access granted	Black box

Conclusion: Error guessing finds realistic attacks and path analysis guarantees that all lockout and failure routes are exercised, so brute-force weaknesses are removed.

Question 12 *(Bloom's Level 5)*

An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.

Answer – E-learning Platform – Tier Upgrade on Expiry Day

- Errors appear when a user upgrades exactly on the expiry date, so the testing criteria did not treat the expiry day as a separate case.
- ECP on dates gives four classes: not started, active, expiry day and expired. Subscription types (Free, Basic, Premium) form a second partition; test cases combine the two.
- Boundary dates are expiry - 1, expiry and expiry + 1, because the failure sits at the edge between 'active' and 'expired'.
- Rule assumed: a subscription stays valid until 23:59 of the expiry date, and an upgrade on that day continues without losing access.

Table 12.1 – ECP and BVA for subscription dates

Date class	Range	Boundary test value	Expected access
Not started	Today < start date	Start - 1	No paid access
Active	Start <= today < expiry	Start, expiry - 1	Full plan access
Expiry day	Today = expiry date	Expiry	Plan access until 23:59
Expired	Today > expiry date	Expiry + 1	Falls back to Free

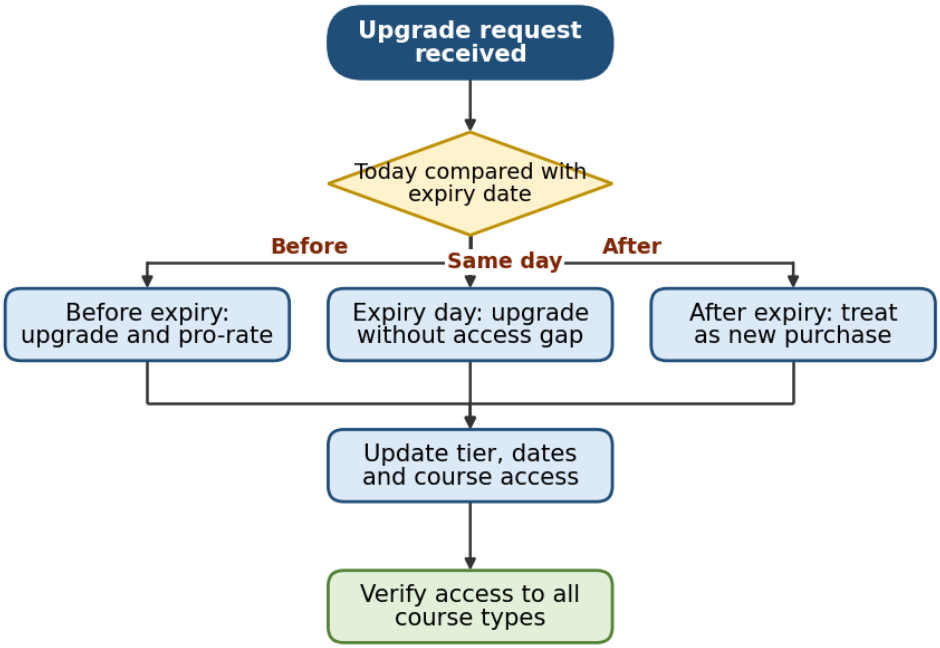


Figure 12.1 – Upgrade request handling by date class

Table 12.2 – Upgrade test cases

TC	Current plan	Upgrade to	Date	Expected result
T1	Free	Basic	Any date	Basic access starts
T2	Basic	Premium	Expiry - 1	Premium, pro-rated
T3	Basic	Premium	Expiry day	Premium, no access gap
T4	Basic	Premium	Expiry + 1	New purchase, Premium
T5	Premium	Basic	Expiry day	Downgrade at renewal
T6	Free	Premium	Expiry day of trial	Premium access, all course types

Conclusion: Treating the expiry day as its own equivalence class and testing the dates on either side makes the plan transitions complete and seamless.

Question 13 *(Bloom's Level 5)*

A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.

Answer – Smart-Home Lighting Controller – Branch and Condition Coverage

- Statement coverage can be 100% while an entire false outcome of a decision, an unused switch case or a failing Boolean condition is never tested; for hardware control these hidden paths can send a wrong command to the device.
- Assumed logic: switch (mode) with DAY, NIGHT, AUTO and default; inside AUTO the dimming decision is D = A AND (B OR C) with A = light level below limit, B = motion detected, C = manual override.
- Improved white box strategy: branch coverage for every case and decision outcome, condition coverage for each of A, B and C, and MC/DC style tests (4 tests for 3 conditions).
- This reduces hardware-software integration risk because every command the software can send has been exercised.

Table 13.1 – Risk of relying only on statement coverage

Criterion	What it checks	Risk if used alone
Statement	Each line executed once	Missed false outcomes and untested cases
Branch	Each decision true and false	Individual conditions still hidden
Condition	Each of A, B, C true and false	Decision outcome may not change
Branch + Condition (MC/DC)	Each condition changes the decision	Lowest risk

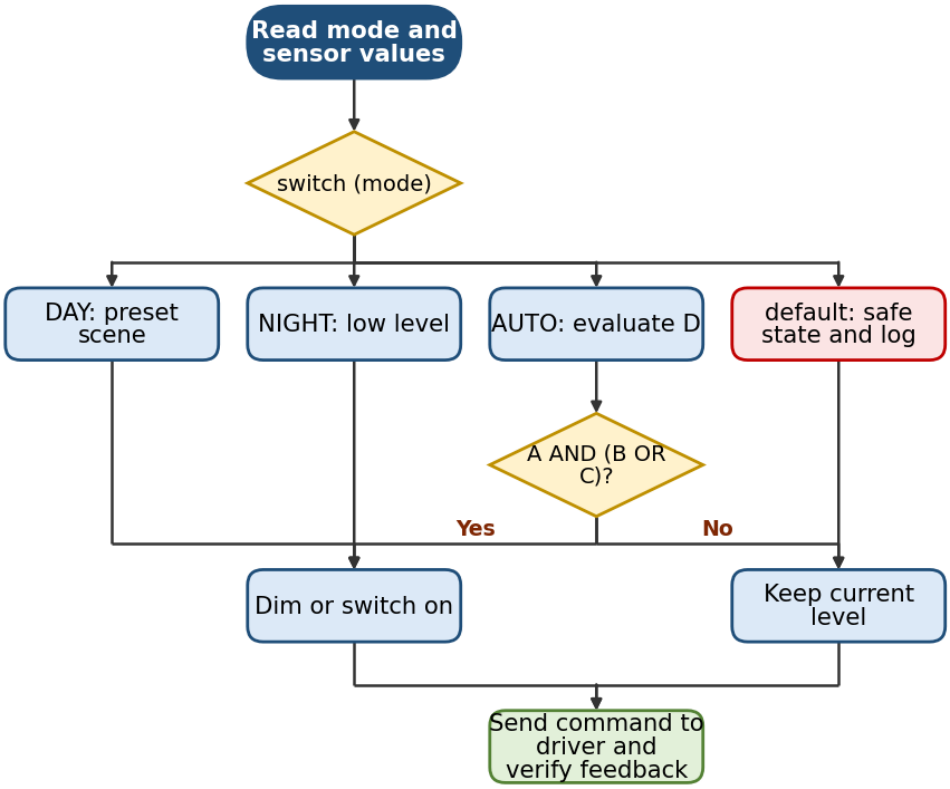


Figure 13.1 – Lighting control logic with switch cases

Table 13.2 – Test cases for switch cases and D = A AND (B OR C)

TC	Mode	A	B	C	D	Covers
T1	DAY	–	–	–	–	DAY case
T2	NIGHT	–	–	–	–	NIGHT case
T3	AUTO	1	1	0	1	Base case, D true
T4	AUTO	0	1	0	0	A changes D (T3, T4)
T5	AUTO	1	0	0	0	B changes D (T3, T5)
T6	AUTO	1	0	1	1	C changes D (T5, T6)
T7	Invalid	–	–	–	–	default case, safe state

Conclusion: Branch and condition coverage with MC/DC style tests cover every case and condition of the controller, greatly reducing the chance of a wrong hardware command.

Question 14 *(Bloom's Level 5)*

A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.

Answer – Logistics System – Zero and Negative Weight Inputs

- Input validation tests used only valid weights, so zero and negative values were never entered. Without a check, the system produced wrong or zero shipping costs.

- Robustness BVA extends normal BVA by also testing just outside the valid limits: min - 1, min, min + 1, nominal, max - 1, max, max + 1, plus extreme invalid values.
- Assumed valid weight: 0.1 kg to 50 kg. Zone and delivery speed are combined with the weight cases.
- Negative scenarios are needed because real users and integrations send bad data; the system must reject it safely instead of calculating a wrong price.

Table 14.1 – Robustness BVA test cases (valid weight 0.1 – 50 kg)

TC	Weight (kg)	Class	Expected result
TC1	-5	Invalid – negative	Rejected with error message
TC2	0	Invalid – zero	Rejected with error message
TC3	0.09	Just below minimum	Rejected
TC4	0.1	Minimum	Cost calculated
TC5	0.11	Minimum + 1	Cost calculated
TC6	25	Nominal	Cost calculated
TC7	49.9	Maximum - 1	Cost calculated
TC8	50	Maximum	Cost calculated
TC9	50.01	Just above maximum	Rejected or freight quote
TC10	abc / blank	Invalid – non-numeric	Rejected

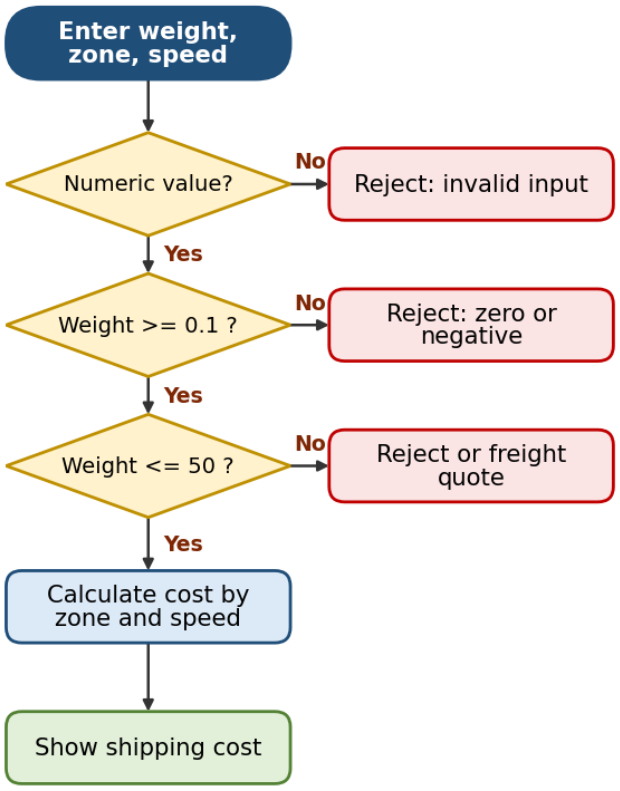


Figure 14.1 – Weight validation and cost calculation flow

Table 14.2 – Why negative scenarios are necessary

Negative scenario	Example	Risk if untested
Zero weight	0 kg	Free shipping or a division error
Negative weight	-5 kg	Negative price or credit to customer
Non-numeric input	'abc'	Crash or wrong calculation
Extremely large weight	99999 kg	Overflow, unrealistic quote
Invalid zone / speed with valid weight	Zone 0	Silent default price

Conclusion: Robustness BVA with negative scenarios proves that invalid weights are rejected cleanly, which keeps pricing correct and the system stable.

Question 15 *(Bloom's Level 5)*

A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.

Answer – Medical Imaging Software – Prime Data Sizes and Basis Path Testing

- The parallel module splits n items among T threads: chunk = n / T, remainder = n mod T, and the first 'remainder' threads take one extra item. For a prime n (greater than T) the remainder is never zero, so this rarely used path always runs.
- Random testing mostly produced sizes such as 16 or 1024 that divide evenly, so the remainder path was almost never tested. Randomness gives no guarantee that a specific mathematical case is reached.
- Predicates: n valid (1), thread loop (1), t < remainder (1), element loop (1) = 4, so V(G) = 5 independent paths; basis path testing needs one test for each.
- Prime sizes (3, 17, 7919) are then used as data for the remainder paths, ensuring mathematical accuracy and stable behaviour across thread counts.

Table 15.1 – Random testing versus structured basis path testing

Aspect	Random testing	Basis path testing
Reaches remainder path	Only by chance	Always – designed test
Repeatable	No	Yes
Coverage measure	None	V(G) paths covered
Prime-size defects	Missed	Found
Suitability for maths code	Low	High

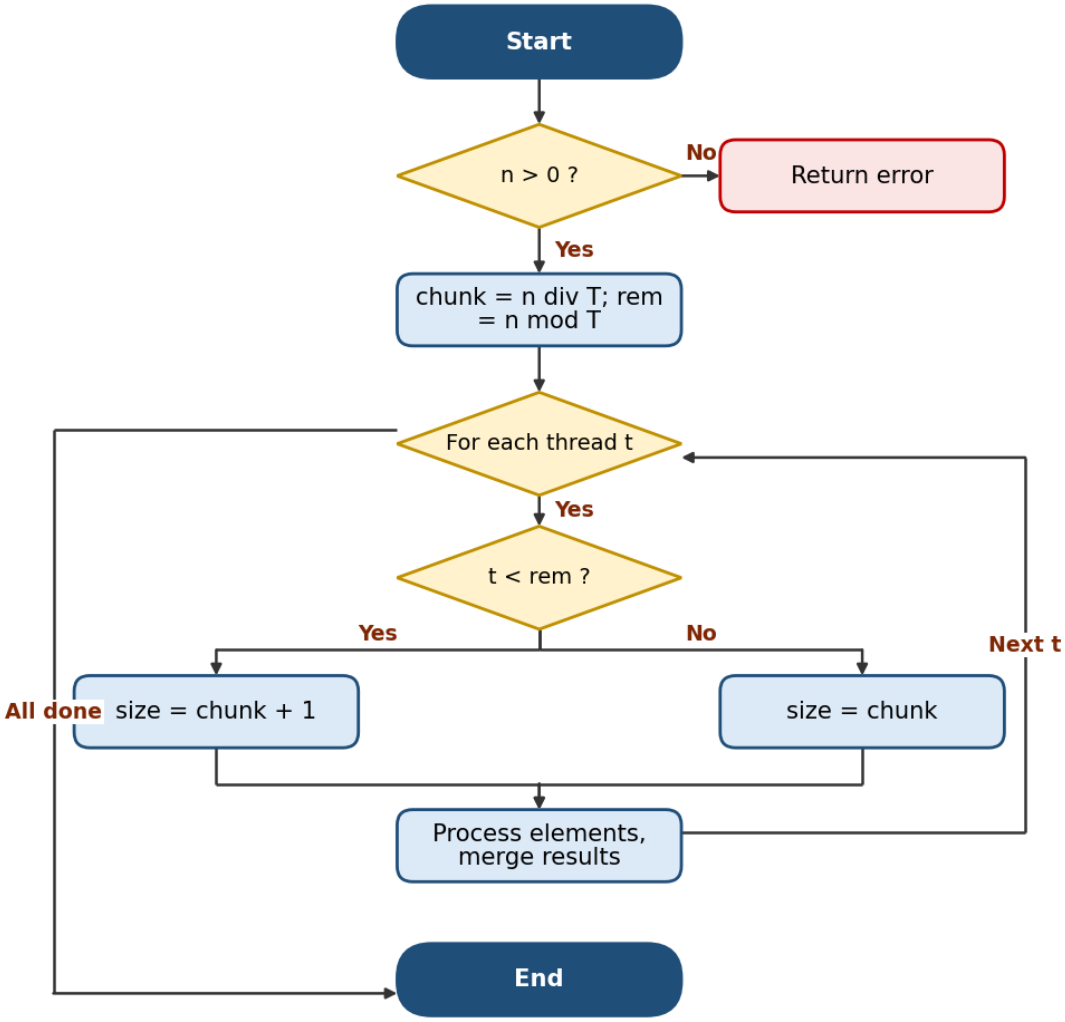


Figure 15.1 – Control flow of the parallel partitioning module

Table 15.2 – Independent paths (V(G) = 5) and test data (T = 4 threads)

Path	Scenario	n	Expected
P1	Invalid size	0	Error
P2	Evenly divisible, remainder = 0	16	Equal chunks
P3	Prime size, remainder > 0	17	One thread gets extra item
P4	n smaller than threads, chunk = 0 (prime)	3	3 threads get one item
P5	Large prime, many iterations	7919	Exact total, no lost items

Conclusion: Basis path testing guarantees that the prime-size remainder logic is executed and checked, which ensures correct results and stable behaviour in the imaging pipeline.